Document No. SPV2-DS-001 Revision A (Draft)

SKETCHPAD VERSION 2

DETAILED SPECIFICATION

Data, Interfaces, Algorithms, Acceptance Tests, Risks and Questions for Analysis

Prepared for	Barry Kingsbury, owner; and the outside analysts he sends it to
Prepared by	Claude (Anthropic), in working sessions with the owner
Date	2 November 2026
Companion to	SPV2-FS-001 Revision B, the functional specification (37 pp.)
Baseline reviewed	Repository `onbostonharbor1/sketchpad`, branches `claude/v1-finish`, `claude/v1-errors`, `claude/ideabook-scripts`
Status	Draft for analysis. Nothing in this document has been built. Section 20 lists the questions the owner most wants an analyst's view on.

Rev	Date	Description
A	2 Nov 2026	First issue, for outside analysis.

CONTENTS

1.0 ABOUT THIS DOCUMENT 5
1.1 Purpose 5
1.2 How to read it 5
1.3 Conventions 5
1.4 What is settled and what is not 5
2.0 CONTEXT AND CONSTRAINTS 6
2.1 The owner and the work 6
2.2 History in brief 6
2.3 Constraints 6
2.4 Scale 7
3.0 ARCHITECTURE 8
3.1 Layers and call rules 8
3.2 Folders 9
4.0 THE KERNEL 10
4.1 Types 10
4.2 Makers 10
4.3 Shapers 11
4.4 Connectors 12
4.5 Composite makers 12
4.6 Parametric sampling 13
4.7 Finders 13
4.8 Acceptance tests for the kernel 14
5.0 GENERATORS AND SETTINGS 15
5.1 The generator file 15
5.2 Settings schema 15
5.3 Acceptance tests 16
6.0 DOCUMENTS, THE POINT TABLE AND WELDS 17
6.1 Document format 17
6.2 Evaluation 18
6.3 Acceptance tests 19
7.0 COMMANDS, UNDO AND SESSION 20
7.1 The command contract 20
7.2 Session 20
7.3 Acceptance tests 21
8.0 STORAGE, CATALOG AND THE NODE SERVICE 22
8.1 The folders are the truth 22
8.2 The Node service 22
8.3 Acceptance tests 23
9.0 THE SCRIPT HOST AND THE LEGACY ADAPTER 24
9.1 Isolation 24
9.2 The legacy adapter 24
9.3 Acceptance tests 25
10.0 RENDERING AND OUTPUT 26
10.1 The canvas renderer 26
10.2 Physical output 26
10.3 Acceptance tests 26

11.0 THE COMMANDS TABLE AND THE SCREENS 28
11.1 Three levels, fixed places 28
11.2 Acceptance tests 28
12.0 ERROR HANDLING 29
12.1 Acceptance tests 29
13.0 EXPLORATION GENERATORS AND GEOMETRY ALGORITHMS 30
13.1 Lattice with shared points 30
13.2 Symmetry repeat 30
13.3 Component groups 30
13.4 Roulettes 30
13.5 Templates 30
13.6 The pursuit engine 31
13.7 Intersections 31
13.8 Image to thread (later) 32
13.9 Acceptance tests 32
14.0 JOURNALS, RUN HISTORY, NOTES AND LEARN 33
14.1 The journal file 33
14.2 Run history 33
14.3 Notes 34
14.4 Learn 34
14.5 Acceptance tests 34
15.0 SWEEP 35
15.1 Acceptance tests 35
16.0 REGRESSION, CHECKS, ROBUSTNESS AND IDEABOOK HINTS 36
16.1 The regression harness 36
16.2 Other checks 36
16.3 Robustness 37
16.4 Ideabook hints 37
17.0 NON-FUNCTIONAL REQUIREMENTS 38
18.0 MIGRATION FROM VERSION 1 39
18.1 Figure files (format 1) 39
18.2 Draw objects 39
18.3 Scripts, hints and approved copies 39
19.0 CONSTRUCTION AND TEST PLAN 40
20.0 RISKS AND QUESTIONS FOR ANALYSIS 42
20.1 Risks 42
20.2 Questions for analysts 42
20.3 What an analyst should not need to settle 43
21.0 REQUIREMENT INDEX 44

LIST OF FIGURES

Figure 1. Structure chart: the layers and who may call whom 8
Figure 2. Makers: the same path made several ways 11
Figure 3. Shapers applied to one path 11
Figure 4. Connectors: four ways of joining the same points 12
Figure 5. A weld: two handles naming one point 18
Figure 6. An attachment: a point found on another layer's path 18

Figure 7. Evaluation order follows attachments 19
Figure 8. Undo: snapshots and a position 20
Figure 9. Session, document and history kept apart 21
Figure 10. Adding an item by saving a file 23
Figure 11. The ingestion ladder: from a foreign script to a generator 24
Figure 12. The screen: where each level of command lives 28
Figure 13. Checkpoints and their order 40

1.0 ABOUT THIS DOCUMENT

1.1 Purpose

This document is the detailed specification of Sketchpad Version 2 ("V2"). It expands every requirement of the functional specification, SPV2-FS-001 Revision B ("the FS"), to the level at which it could be built and tested: the exact data formats, every interface between parts, the algorithms, the limits, the error cases, and an acceptance test for each requirement. It is written to be read by people who have not seen Sketchpad, so that they can analyse the design, find its weaknesses, and say where it should change before construction starts.

Where the FS says *what*, this document says *how exactly* and *how we will know*. Where the two differ, the FS governs and this document is in error; please report it.

1.2 How to read it

- Section 2 is the context: who the owner is, what he makes, and the constraints the design must respect. Read it first; much of the design follows from it.
- Sections 3 to 16 specify the parts, in the order data flows through them: the kernel, generators and settings, documents, commands, session, storage, the script host, rendering and output, and then the cross-cutting parts (commands table, errors, Sweep, journals, regression, robustness).
- Section 17 gives the non-functional requirements; section 18 the migration from V1; section 19 the construction and test plan; section 20 the risks and the questions for analysts; section 21 the requirement index.

1.3 Conventions

As in the FS: "shall" is a requirement, "should" a recommendation, "may" an option. Requirement numbers are those of the FS (FR-s.n). Detailed requirements added here are numbered DR-s.n by section of this document. Acceptance tests are AT-s.n. Risks are K-n; analyst questions are Q-n. Data structures are given as JSON with comments (`//`), which JSON itself does not allow; the files themselves carry none. Interfaces are given as JavaScript signatures. Lengths are in millimetres unless said otherwise; canvas coordinates are in pixels, y downward.

1.4 What is settled and what is not

Decisions recorded with the owner are D-1 to D-15 in the FS (section 4.2). Open items are O-1 to O-16 (FS section 12). This document does not settle open items; where a design depends on one, both branches are specified and marked.

2.0 CONTEXT AND CONSTRAINTS

2.1 The owner and the work

The owner is a retired technical writer and an old-time Unix maintenance programmer (Xerox, Sun, Stellar, TotalView, NobleNet). Sketchpad is the first program he has written in twelve years; much of it is relearning. He reads every line he keeps, so the code must be plain and readable rather than clever. He works on Windows in VS Code, reviews from an iPad, and uses AI assistants to write code under his rules. Earlier assistants failed him in two ways the design must guard against: deleting code unrelated to the change in hand, and taking a shortcut to the result instead of using the pattern that already existed.

He makes string art: thread wound between nails driven into a board, the threads' envelope forming curves. The classic construction is the *corner stitch*: two lines of nails meeting at a corner, nail i of one joined to nail i of the other, which draws a parabola. His aim is original work rather than formulaic craft. Sketchpad has two purposes of equal standing (FS 1.2):

1. **Templates.** A printable, true-scale point template for driving the nails.
2. **Sketching.** A place to explore designs cheaply by working with how objects are represented and how they relate. Making exploration cheap is a major function (D-8).

2.2 History in brief

Stage	What it was	What it taught
StringThing (to late 2025)	One 4,900-line HTML file: a single `StringThing` settings object with about 30 defaulted fields, 42 numbered tests switched on by uncommenting a line, and the numbered Figures of the owner's ideabook.	Separate the definition of a drawing from how it is drawn. Declare little, default everything, draw whatever is asked (D-15). A point may be referred to by number (`new Point(n)`), the seed of the V2 point table.
Manager (2 weeks)	A manager of StringThing instances.	A dead end: managing more copies of a design that cannot grow does not help.
Sketchpad v.1	A UI model from the owner's workflow (Draw, Patterns, Gallery, Utilities), a draw registry, manifests.	The tabs have barely changed since.
Sketchpad v.2 ("V1" in these documents)	Subsystems, categories, Home; then Figures (layers, discs, welds, transforms), print aids, magic points; then error reporting, regression tests and Ideabook hints (November 2026).	The UI held; the drawing side, never rethought, is straining (FS 3.3).

The owner's own numbering calls the rebuild Sketchpad 3 (O-11). These documents keep "V1" and "V2".

2.3 Constraints

DR-2.1	Plain JavaScript ES modules. No framework, no build step, no transpiler, no TypeScript (A-2). A dev server (Vite) may serve files; the code must also run from a plain static server for read-only review.
DR-2.2	A local Node service (one process, one port) performs every file write. The browser never writes files.
DR-2.3	Windows is the primary platform; macOS and Linux shall also work (the owner may work from a borrowed Mac or a Linux USB drive). Paths shall be handled with the platform's path module, never by string joining with "/".
DR-2.4	Works offline (FR-6.18.7). No CDN at run time.

DR-2.5	Readable code: one file per idea, a header comment saying what it does (read by the Notes popup), names that say what they are, no cleverness for its own sake. A function longer than about 60 lines should be split unless splitting hides the idea.
DR-2.6	No implicit globals. Shared state is declared in a state object (FS 6.7).
DR-2.7	The iPad is for review only (D-2): reading documents and the running app; no authoring.
DR-2.8	Everything is in git; small, described commits; a branch per phase, reviewed by the owner before merge.

2.4 Scale

Item	V1 today	V2 design point
Owner's code (excluding scripts and vendored libraries)	about 27,000 lines	should be smaller for the same function
Draw objects (generators)	14	50, added by file
Pattern and gallery scripts	185	500
Ideabook images / hints	about 650 / 282	2,000 / 1,000
Layers in one figure	up to about 20	100
Points (nails) in one figure	a few hundred	5,000
Lines drawn for one figure	up to about 20,000	200,000 within the time budget of DR-17.1

3.0 ARCHITECTURE

3.1 Layers and call rules

V2 has five layers. A layer may call only the layers below it. Data, not calls, crosses upward.

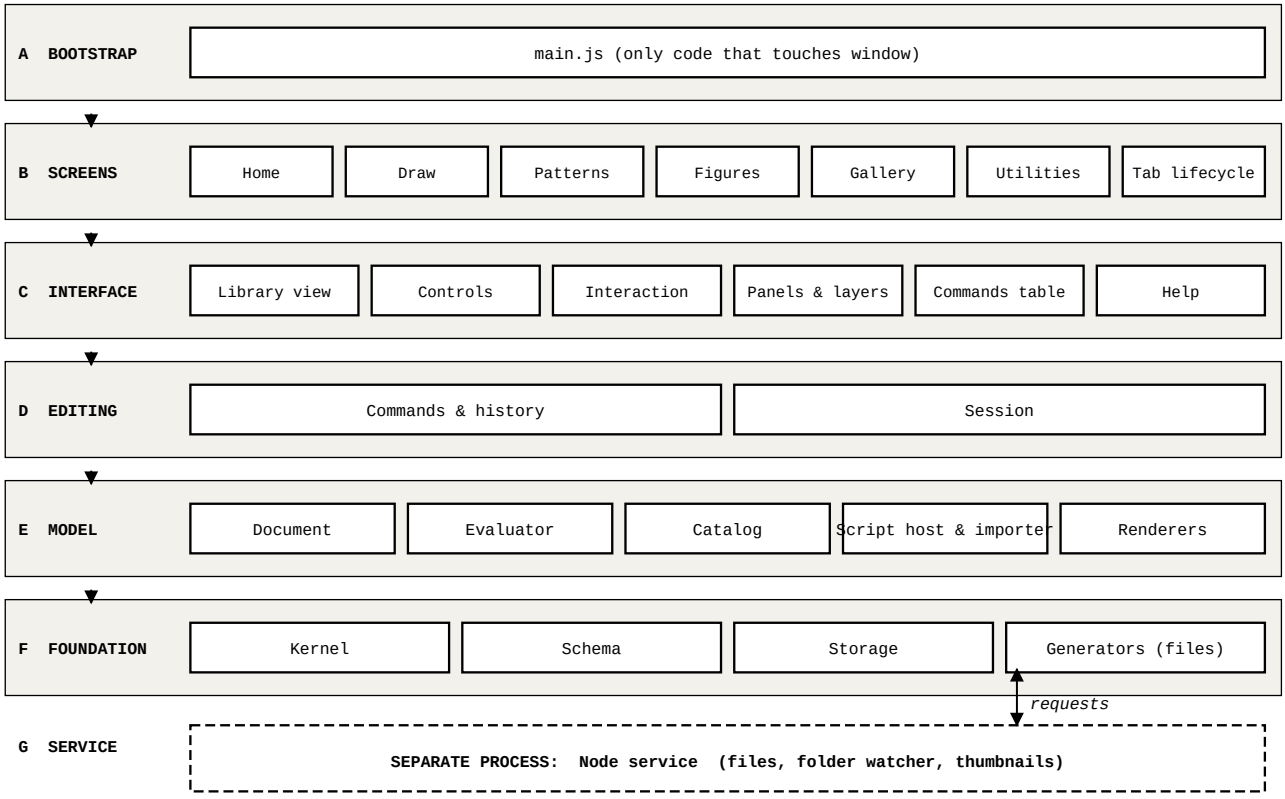


Figure 1. Structure chart: the layers and who may call whom

Layer	Contents	May call	Must not
5 Screens	Tabs, panels, menus built from the commands table, the canvas element	4, 3 (read only), 2 (render)	change a document except through a command
4 Commands	The commands table; each command takes a document and returns a new one	3, 1	touch the page or the canvas
3 Documents and session	Document model, point table, evaluation, session state, history	1	paint, or know about tabs
2 Renderers and output	Canvas, SVG, PDF, nail list, thumbnails	1 (types only)	evaluate geometry, read session
1 Kernel and generators	Paths, makers, shapers, connectors, finders; generator files	nothing above	use the page, the canvas, time, randomness without a seed, or any state

DR-3.1	A check (part of the import check, FR-6.17) shall read every module's imports and fail if a module imports from a layer above its own. Layer is given by folder.
DR-3.2	The Node service is outside the layers; only the storage module of layer 3 and the catalog may call it, through named requests (section 9).

3.2 Folders

```
sketchpad/
  kernel/        paths.js, makers/, shapers/, connectors/, finders/      layer 1, pure
  generators/    one file per generator (circle.js, parabola.js, lattice.js ...)
  templates/     one file per template (rectangle.js, ruler.js ...)
  documents/     model.js, points.js, evaluate.js, migrate.js, checks.js   layer 3
  session/       session.js, history.js
  commands/      table.js, one file per family (layers.js, points.js, sweep.js ...)
  render/        canvas.js, svg.js, pdf.js, nails.js, thumbnails.js       layer 2
  screens/       one folder per tab, plus panels/ and menus.js            layer 5
  host/          the script host (iframe page and adapter)
  service/       the Node service (requests, watcher, paths)
  library/       the owner's content: figures/, scripts/, ideabook/, help/
  journal/       journals, one file per item (section 14)
  tests/         regression/, kernel/, service/, screens/
  vendor/        outside libraries, pinned
```

Library content is kept apart from code so that the code can be replaced, and the content kept, without mixing the two.

4.0 THE KERNEL

4.1 Types

```
// A point. z defaults to 0 (D-11).
Point   = { x: number, y: number, z?: number }

// A path: points in order, and whether the last joins the first.
Path    = { pts: Point[], closed: boolean, name?: string }

// A stitch: one thread, from a to b. src names where each end came from.
Stitch  = { a: Point, b: Point, src?: { pathA: string, i: number, pathB: string, j: number } }

// What a generator returns. Nothing in it is a function; it can be saved as JSON.
Geometry = {
  paths:    { [name: string]: Path },     // the infrastructure: where nails can go
  stitches: Stitch[],                     // the threads
  points:   { [name: string]: Point },    // named points: handles, features
  bounds:   { x0, y0, x1, y1 },
  style:    { color, lineWidth, alpha? }  // per stitch overrides allowed: Stitch.style
}
```

DR-4.1	Kernel functions shall take values and return new values; they shall never change their arguments. A test shall freeze every argument (`Object.freeze`, deeply) and call each kernel function with it.
DR-4.2	Kernel functions shall not read `window`, `document`, a canvas, the clock, or `Math.random`. Randomness comes from a seed argument through the kernel's own generator (DR-4.12).
DR-4.3	Every kernel function shall have defaults for every setting (D-15), and shall return a result for any input of the right type, however odd, except where the result is undefined (a path of zero points cannot be joined), in which case it returns an empty result and a note (DR-4.4), never throws.
DR-4.4	A kernel function may return notes beside its value: `{ value, notes: ["arm 2 has 12 points, arm 1 has 20: joined the first 12"] }`. Notes are shown in the layer's status, not as errors.

4.2 Makers

A maker returns a path. Signature: `maker(settings) → Path`. Settings are listed with their defaults.

Maker	Settings (default)	Result
line	a {0,0}, b {100,0}, n 10	n+1 evenly spaced points, open
circle	center {0,0}, r 100, ry = r, n 24, start -90°, turn 0°, spacing "angle" | "arc"	n points, closed. "arc" spaces evenly by arc length (V1's createNodes), which matters for ellipses
arc	center, r 100, from 0°, to 180°, n 10	n+1 points, open
polygon	center, r 100, sides 3, perSide 10, turn 0°	points along the edges, corners once each, closed
star	center, R 100, r 40, n 5, turn 0°	2n corners, tips at R, notches at r, closed
zigzag	center, width 500, height 150, teeth 6	2·teeth+1 corners, open
spiral	center, r0 10, r1 200, turns 3, n 120, kind "archimedean" | "log"	n+1 points, open
parametric	fx(t), fy(t) as expressions, t0, t1, n, sampling "even-t" | "even-length" | "adaptive"	see 4.6

roulette	center, R 150, r 52 (negative: outside), d 60, turns 13, steps 600, dChange 0 per turn	hypotrochoid or epitrochoid; dChange varies the pen distance as it draws (FR-6.19.4)
points	pts: Point[]	the given points (for hand-placed nails)

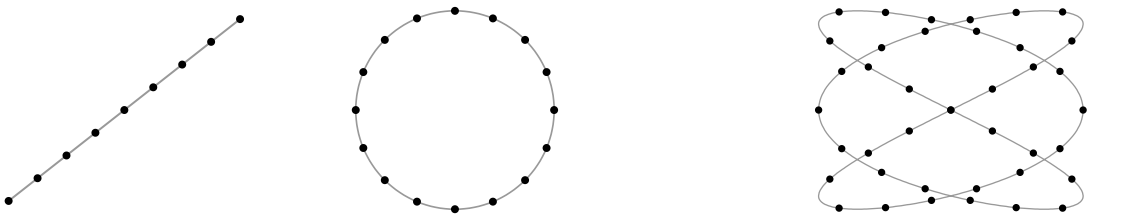


Figure 2. Makers: the same path made several ways

4.3 Shapers

A shaper takes a path and returns a path: `shaper(path, settings) → Path`.

Shaper	Settings (default)	Effect
trim	from 0, to = length	keep a run of points
reverse	—	run backwards
rotateStart	k 0	start at point k (closed paths)
resample	n, by "count" | "length"	new points spaced evenly along the old
jitter	amount 0, seed 1, mode "radial" | "both"	repeatable wobble (FR-6.2.4)
bias	amount 0	crowd points toward one end (V1's spacingBias)
transform	angle, tx, ty, scale, about	move, turn, scale
mirror	axis (two points)	reflect
simplify	tolerance 0.25 px	drop points that change nothing (Ramer–Douglas–Peucker)

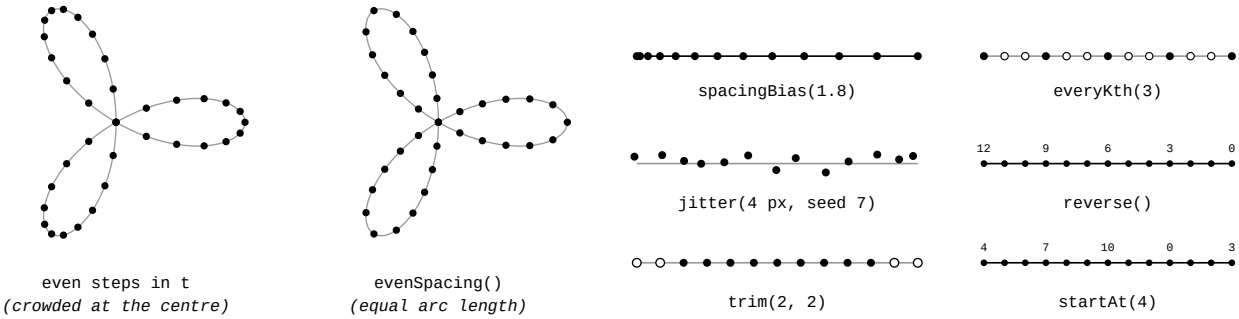
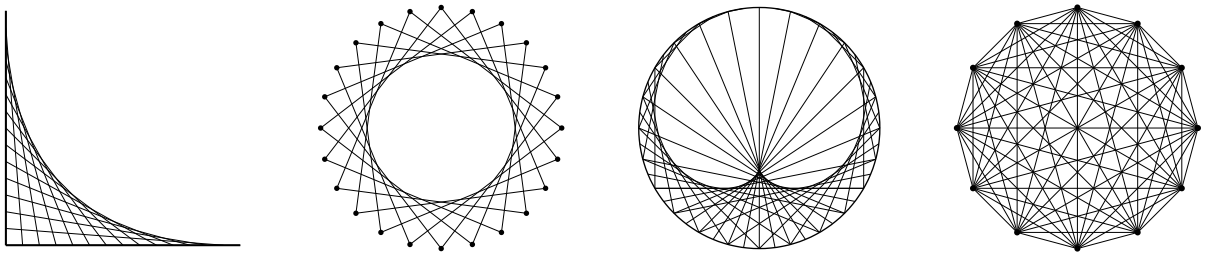


Figure 3. Shapers applied to one path

4.4 Connectors

A connector takes one or two paths and returns stitches: `connector(a, b?, settings) → Stitch[]`. The family is pluggable (FR-6.2.5a): each connector is a file in `kernel/connectors/`. Notation: a_i is point i of path a, n_a its length.

Connector	Rule	Settings (default)	Notes
pairwise	$a_i – b_{i+offset}$ for i = from, from+every, … < to	offset 0, every 1, from 0, to n_a, reverse false (b run backwards), wrap false	V1's `stitcher` is offset 0; past b's end without wrap: stop, with a note
corner	pairwise on line(a → c, n) and line(c → b, n)	a, c, b, n 15	the classic curve stitch
skip	$a_i – a_{(i+s) \bmod n}$	s 1, wrap true, from, to	replaces V1's three chord functions
growingSkip	$a_i – a_{round(i + s0 + i·g) \bmod n}$	s0 30, g 0.4	curls and teardrops (14 of the owner's boards)
timesTable	$a_i – a_{round(i·k) \bmod n}$, skipping j = i	k 2 (negative and fractional allowed)	k=2 cardioid, 3 nephroid, −2 deltoid, −1 parallel chords
allPairs	$a_i – a_j$, i < j	—	Mystic Rose; n(n−1)/2 stitches: budget-checked
ring	$a_i – a_{i+1}$	closed	outlines
mirror	$a_i – a_{n−1−i}$	—	
linked	among several paths, by link style pairwise | ring | all	style "pairwise"	Linked Circles
fan	every a_i to one point p	p	bursts, cones, wheels



`pairwise(arm1, arm2)` *the parabola* — `skip(s = 7)` *chords in a circle* — `multiply(k = 2)` *the cardioid (Cycloid)* — `allPairs()` *the mystic rose*

Figure 4. Connectors: four ways of joining the same points

DR-4.5	Each stitch shall carry `src`: the path names and indices of its two ends. Printing (nail numbering, colour cycle FR-6.11.9) and hit-testing read it.
DR-4.6	When two paths of different lengths are joined pairwise, the default shall be to join the shorter's length and note it; a setting `match: "resample"` resamples the longer to the shorter (FR-6.2.6).

4.5 Composite makers

Built from the above, and proven by the 282 Ideabook hints (FR-6.2.5b). Each returns Geometry.

Composite	Construction

starOfCorners(center, r, n, steps, inner 0, twist 0, turn 0)	tips t_k on a circle of radius r; for each k a corner stitch from t_k to t_{k+1} bending round c_k, the centre when inner = 0, otherwise a point on a circle of radius inner at the half-angle between the tips, moved round by twist. n=4, inner 0: the four-pointed star; inner > 0: long thin points; twist: pinwheel blades.
polygonCornerStitch(corners, steps, full false)	at every corner a corner stitch between the mid-points of its two edges, or with full the whole edges (corner to corner), leaving the round or elliptical holes of many boards.
segmentStitch(a1, a2, b1, b2, steps)	pairwise on line(a1→a2) and line(b1→b2).
zigzagStitch(path, steps, colors)	a corner stitch at every inner corner of a path.
offsetRings(center, r, n, rings, shrink, offset, drift)	rings of nails each pairwise-joined to the next with an offset, the centres drifting: nested spirals.
pursuit(start, …)	section 13.6.

4.6 Parametric sampling

V1 samples a parametric curve at Loops × 60 even steps in t (4,500 points at the default), all drawn and all searched (I-3). V2 samples adaptively.

```
sampleAdaptive(f, t0, t1, tol = 0.25 px, maxDepth = 14):
  start with 64 even steps in t
  for each interval [ta, tb]:
    m = f((ta+tb)/2);  e = distance from m to the chord f(ta)-f(tb)
    if e > tol and depth < maxDepth: split the interval and recurse
  then simplify (RDP, tol) and, if a nail count n is asked for, resample by length to n
```

DR-4.7	For the default Parametrics settings, the sampled path shall have at most 720 points (V1's infrastructure limit) and draw within 0.5 px of the 4,500-point curve.
DR-4.8	"Nails" (infrastructure) and "drawing" (the visible curve) may use different samplings of the same curve; a nail count is the owner's choice, a drawing tolerance is not.

4.7 Finders

A finder takes geometry and returns named points: `finder(geometry, settings) → { [feature]: Point }`.

Finder	Feature names	Rule
ends	`start <path>`, `end <path>`	first and last points
middles	`mid <path>`	half way by length
extremes	`top|bottom|left|right <path>`	by coordinate
lobeTips	`tip <path> #k`	a point farther from the curve's centre than its neighbours by at least 1.5 px within 3 points either side (V1 G-12), numbered by angle from the top, clockwise
cusps	`cusp <path> #k`	a turn sharper than 100° between consecutive segments
intersections	`cross <pathA>.<pathB> #k`	section 13.7

DR-4.9	Feature numbering shall be stable under small changes: numbered by angle about the path's centroid from straight up, clockwise, so that moving a handle a little does not renumber.

DR-4.10	Where two features coincide within 0.5 px, the more telling wins: intersection, cusp, tip, middle (V1 G-11).
DR-4.11	A curve's intersections with itself shall be offered only when asked for (they run to thousands on a dense curve, V1 G-10).
DR-4.12	The kernel's random source shall be a small seeded generator (Park–Miller, 16807 mod $2^{31}-1$), so the same seed gives the same numbers in every browser.

4.8 Acceptance tests for the kernel

AT-4.1	circle(n 36) + timesTable(k 2) gives 35 stitches whose envelope is a cardioid (checked against the cardioid's equation within 1 px at 36 sample angles).
AT-4.2	corner(a, c, b, n 15) gives 16 stitches; stitch i joins a + (c−a)·i/15 to c + (b−c)·i/15.
AT-4.3	jitter with seed 7, then seed 8, then seed 7 again: the first and third results are identical to the last bit.
AT-4.4	Every kernel function, called with frozen arguments, does not throw and does not change them (DR-4.1).
AT-4.5	pairwise with arms of 12 and 20 points joins 12 and returns one note naming both lengths.
AT-4.6	Feature numbers of a rose's lobe tips do not change when the rose is turned by 1° (DR-4.9).
AT-4.7	allPairs on 400 points (79,800 stitches) completes inside the budget of DR-17.1 or stops with the budget message (FR-6.18.2).

5.0 GENERATORS AND SETTINGS

5.1 The generator file

A generator is one file. It declares what it is and what settings it takes, and has one function that turns settings into geometry. It replaces both V1's Draw registry objects and its `scriptInfo` scripts.

```
// generators/parabola.js
/* Parabola: two arms meeting at a corner, nail i of one joined to nail i of the other.
   The classic corner stitch. Settings: the three corner points, the number of steps. */
export const meta = {
  kind: "generator", id: "parabola", version: 2,
  title: "Parabola", tags: ["fundamental"], notes: "The corner stitch."
};
export const params = {
  a:      { type: "point",  default: { x: 100, y: 100 }, handle: true, label: "Arm 1 end" },
  corner: { type: "point",  default: { x: 100, y: 400 }, handle: true },
  b:      { type: "point",  default: { x: 400, y: 400 }, handle: true, label: "Arm 2 end" },
  steps:  { type: "int",    default: 15, min: 2, max: 200 }
  // colour, line width, rotate, scale: the common set (DR-5.6), not declared here
};
export function build(s, k) {            // s: settings, k: the kernel
  const arm1 = k.line({ a: s.a, b: s.corner, n: s.steps });
  const arm2 = k.line({ a: s.corner, b: s.b, n: s.steps });
  return k.geometry({ paths: { arm1, arm2 }, stitches: k.pairwise(arm1, arm2) });
}
export function migrate(old, fromVersion) { /* settings from an older version */ return old; }
```

DR-5.1	`build` shall be a pure function of its settings and the kernel. It shall not keep state between calls; every call starts fresh (this is what retires V1's shared singletons, P-2).
DR-5.2	A generator may export `drag(settings, handleName, newPoint) → settings` when dragging a handle must change more than that handle (a midpoint handle that moves both ends). Without it, dragging sets the handle's setting.
DR-5.3	A generator shall be registered by saving its file; the catalog reads `meta` and `params` without running `build`, and checks the schema once (FR-6.14.4). A failing check shows the generator greyed in its list with the reason.
DR-5.4	`version` is a whole number. A document records the version each layer was made with; on open, `migrate` brings older settings forward (FR-6.5.6).
DR-5.5	Generators are judged by DR-2.5 (readable) above all: a generator should fit on one screen; anything longer is built from kernel parts.

5.2 Settings schema

Type	Fields	Control	Value
int	default, min, max, step 1	slider with a number box	whole number
number	default, min, max, step	slider with a number box	number
angle	default (degrees), min −360, max 360	slider and dial	degrees
color	default "#rrggbb"	colour picker	"#rrggbb"
choice	default, options [..]	list, or buttons when 4 or fewer	one option

bool	default	checkbox	true / false
text	default	text box	string
point	default {x, y}, handle true|false	a handle on the canvas, and two number boxes	{x, y, z}
points	default [..], handles true|false	handles; add and remove	Point[]
pathRef	default null	choose another layer's path	{ layer, path }
seed	default 1	number box with a "new seed" button	whole number

Optional fields on any type: `label`, `section` (groups settings into a tab of the settings panel), `showIf` (a condition on other settings), `help`.

DR-5.6	The common set, shared by every generator and shown in a standard panel (FR-6.4.9): colour, line width, steps (where meaningful), rotate, scale, opacity. A generator may hide one it does not use (`common: { steps: false }`).
DR-5.7	Every setting's default shall lie inside its limits; the schema check (DR-5.3) enforces it (the V1 registry check found sliders whose settings were not numbers).
DR-5.8	A value from a saved file outside its limits shall be kept, shown with a mark, and used: limits guide the slider, they do not refuse a value (D-15).

5.3 Acceptance tests

AT-5.1	Saving a new generator file makes it appear in Draw's list within 2 s with no restart (FR-6.9.2); a file with a default outside its limits appears greyed with that reason.
AT-5.2	Two Figures layers made from the same generator, one changed, do not affect each other or Draw (P-2).
AT-5.3	A document saved with generator version 1, opened after the generator is at version 2, opens with its settings migrated and draws.

6.0 DOCUMENTS, THE POINT TABLE AND WELDS

6.1 Document format

```
{
  "format": 2,                        // V2's first format; V1 files are format 1 (section
18)
  "createdBy": "sketchpad-v2",
  "title": "Star in a square",
  "units": "mm",                      // "mm" | "in" | "legacy" (V1 pixels)
  "scale": 0.5,                       // mm per document unit; null until set for output
  "nail": { "diameter": 2, "clearance": 3 },
  "nextPoint": 14,                    // point numbers are never reused
  "points": {                         // the point table
    "1": { "x": 100, "y": 100, "z": 0 },                // a free point
    "2": { "on": "L1", "path": "arm1", "t": 0.5 },      // attached by fraction along a path
    "3": { "on": "L2", "feature": "tip curve #3" }      // attached by feature name
  },
  "layers": [
    { "lid": "L1", "generator": "parabola", "version": 2,
      "settings": { "steps": 15 },                      // only settings that differ from
defaults
      "handles": { "a": 1, "corner": 5, "b": 6 },        // handle name to point number
      "transform": { "angle": 0, "tx": 0, "ty": 0 },
      "role": "stitch",                                  // "stitch" | "guide" | "scaffold" |
"image"
      "includeInOutput": true, "visible": true, "color": "#1c7ed6", "legacyNumber": 1 }
  ],
  "order": ["L2", "L1"],              // top first; separate from numbering
  "discs": [ { "point": 2 }, { "point": 3 } ],          // which points are nails shown on
the canvas
  "output": { "paper": "letter", "margin": 10, "overlap": 10, "labels": true, "colourCycle":
false }
}
```

DR-6.1	A document holds no derived values: no drawn geometry, no bounds, no cached positions. Everything shown is evaluated from it (P-3).
DR-6.2	A weld is two handles naming one point number (D-3). There is no weld list to keep in step (P-6).
DR-6.3	An attached point that can no longer be found (its feature vanished) shall keep its last position, be marked, and be listed by the integrity check (FR-6.18.5); it shall not jump to another feature.
DR-6.4	Settings equal to the generator's default are not written, so a changed default reaches old documents unless they set the value.
DR-6.5	Files are written with sorted keys and two-space indent, so a diff in git shows only what changed.

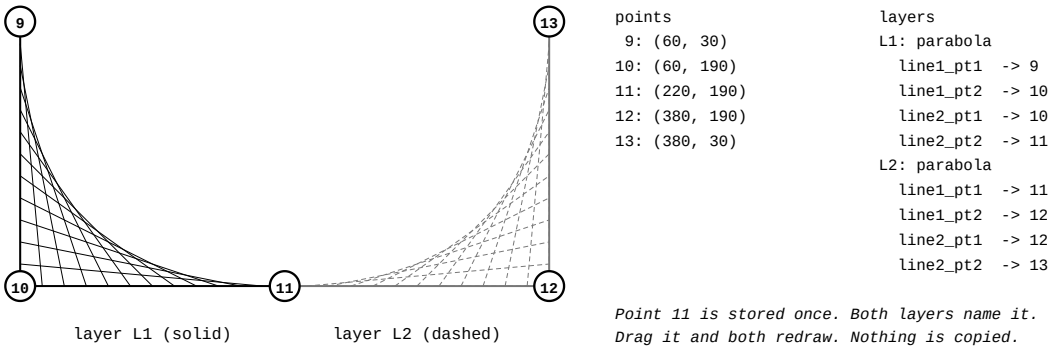


Figure 5. A weld: two handles naming one point

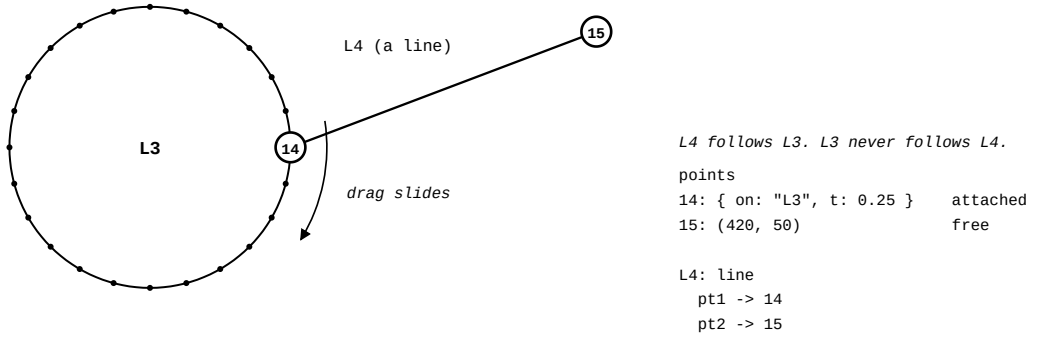


Figure 6. An attachment: a point found on another layer's path

6.2 Evaluation

Evaluating a document gives, for every layer, its geometry, and for every point, its position.

```
evaluate(document):
  graph = layers as nodes; an edge L1 → L2 when a handle of L2 names a point attached to L1
  if the graph has a cycle: report the layers in it; evaluate the rest; mark the cycle's
layers failed
  for each layer in topological order:
    for each handle: position = free point, or the attached point evaluated on its layer
(already done)
    settings = defaults + layer.settings + handle positions
    geometry = generator.build(settings, kernel)       // caught here: a failure marks this
layer only (FR-6.14.3)
    apply layer.transform to geometry
    record geometry; evaluate attached points that lie on it
  return { geometries, positions, notes, failures }
```

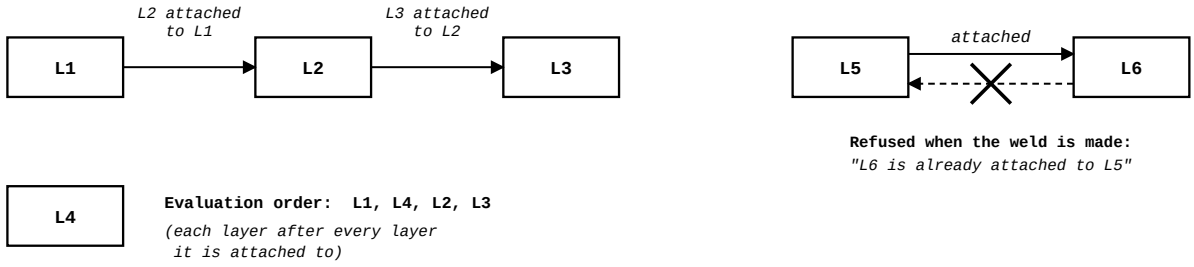


Figure 7. Evaluation order follows attachments

DR-6.6	Evaluation shall be memoised per layer by a key of its inputs (settings, handle positions, transform); a change to one layer re-evaluates it and the layers that depend on it, no others. Target: dragging a handle on a 20-layer figure redraws at 60 frames a second (DR-17.1).
DR-6.7	A cycle of attachments shall be refused at the command that would create it, with a message naming the layers; evaluation's cycle handling is the backstop for hand-edited files.

6.3 Acceptance tests

AT-6.1	Weld two Parabola corners; drag one: both arms follow in the same frame, with no propagation code (DR-6.2).
AT-6.2	Attach a disc to "tip curve #3" of a rose; change the rose's petals from 5 to 7 and back: the disc follows tip #3 throughout and ends where it began.
AT-6.3	Save, close, open: the file is byte-identical when saved again with no change (DR-6.5).
AT-6.4	A hand-edited file with an attachment cycle opens; the cycle's layers are marked and listed; the rest draw.

7.0 COMMANDS, UNDO AND SESSION

7.1 The command contract

```
Command = {
  id: "layers.insert",                // family.name, unique
  label: "Insert object...",
  scope: "tab" | "subtab" | "global",  // section 11 (FR-6.13)
  enabledWhen(ctx) -> boolean,
  whyNot(ctx) -> string,              // shown as the greyed item's tooltip
  run(doc, ctx, args) -> { doc, select? }   // returns a new document; never changes doc
}
```

DR-7.1	Every change to a document is a command (P-7). A command returns a new document; the old one is kept by history unchanged (structural sharing allowed; mutation not).
DR-7.2	History is a list of snapshots with a position (A-3). Undo moves back, redo forward; a new command after an undo drops the redo side. The list keeps at least 200 snapshots or 50 MB, whichever is first, dropping the oldest.
DR-7.3	A drag is one command, recorded when the pointer is released; intermediate positions are drawn but not recorded. A run of nudges with the arrow keys within 1 s is one command.
DR-7.4	A command that throws leaves the document as it was (it never changed it); the error is reported (FR-6.14.3).
DR-7.5	"Changed since saved" is the history position differing from the saved position; selection, mode and view are session, not document, and never mark it changed (P-8).

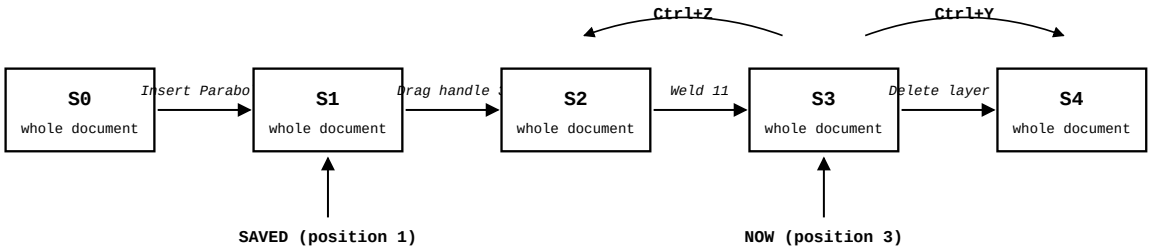


Figure 8. Undo: snapshots and a position

7.2 Session

```
Session = {
  tabs: { [tabId]: { open: documentId[], active: documentId, activeLayer: lid,
                     mode: "lines" | "points", showAll: boolean, hidden: lid[],
                     selected: pointNumber | null, zoom: number, pan: {x, y} } },
  activeTab: tabId
}
```

DR-7.6	Tab switching shall be a pure function of session state: the tab shows what its session says. There is no "needs update", "saved" or "restore" decision chain (P-9).
DR-7.7	The session shall be written to the journal store as it changes (at most once a second) for crash recovery (FR-6.18.3); on start-up after an unclean exit the owner is offered it back.

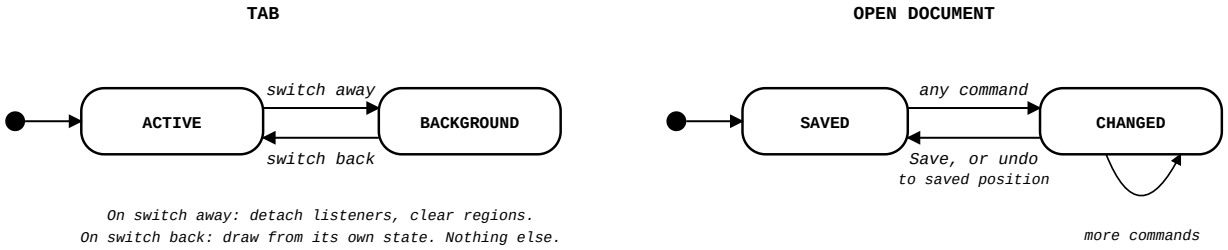


Figure 9. Session, document and history kept apart

7.3 Acceptance tests

AT-7.1	Delete a layer, undo: the layer, its points and its welds are back exactly (document equal to before).
AT-7.2	Select a layer and change mode: the figure is not marked changed.
AT-7.3	Kill the browser tab mid-edit, reopen: the session is offered back with the unsaved document.

8.0 STORAGE, CATALOG AND THE NODE SERVICE

8.1 The folders are the truth

V1 keeps manifests: stored lists of what is in each folder, rebuilt at start-up and checked for missing files (P-10). In V2 the catalog is worked out from the folders and a small metadata file beside each item; nothing else is stored about them.

```
library/scripts/curve-stitch/manyRowsColumns.js          the item
library/scripts/curve-stitch/manyRowsColumns.meta.json   { "status": "favorite", "tags":
[...], "order": 3, "hidden": false }
library/scripts/curve-stitch/manyRowsColumns.png         its thumbnail, made by the renderer
```

DR-8.1	The catalog entry of an item shall be: its id (path from the library root, without extension), kind, category (its folder), title (from its header or `meta`), status and tags (from `.meta.json`), thumbnail, and for scripts the host's report (section 9). It is rebuilt for one item when that item's files change.
DR-8.2	Renaming or moving an item shall be a command that moves its files, its metadata, its thumbnail and its journal together (FR-6.16.2).
DR-8.3	Archiving shall move an item into its category's `archive/` folder; archived items are left out of lists, tests and the catalog, and can be brought back.

8.2 The Node service

One process on one port (5174). Requests are `POST /dispatch` with `{ requestName, payload }`, as in V1. The service also keeps one Server-Sent Events connection open to tell the application that files changed.

Request	Payload	Reply
catalog.list	{ kind?, category? }	{ entries: CatalogEntry[] }
item.read	{ id }	{ text, meta }
item.write	{ id, text, expectedVersion }	{ version } or { conflict: currentVersion }
item.move	{ id, toId }	{ id }
item.archive	{ id }	{ archivedId }
meta.write	{ id, meta }	{}
thumbnail.write	{ id, png (base64) }	{}
journal.append	{ id, entry }	{}
journal.read	{ id, since? }	{ entries }
log.append	{ kind: "errors" | ..., lines }	{ name }
log.list / log.read	{ limit } / { name }	V1's listLogFiles / readLogFile
output.write	{ id, format, bytes }	{ path } (SVG, PDF, DXF beside the item)
editor.open	{ id }	opens the file in VS Code
history.vscode	{ id }	VS Code's saved versions (V1 Script history, kept as a bridge)
status	{}	{ version, root, uptime } (FR-6.18.6)

DR-8.4	Every path in a payload shall be an id relative to the library root; the service resolves it and refuses anything outside the root, any `..` segment, and any absolute path (V1's path-safety module, kept).
DR-8.5	Every write shall go to a temporary file in the same folder and be renamed into place, so a crash leaves the old file whole (V1 already does this for figures).
DR-8.6	`item.write` takes the version the editor read; if the file has changed since (the owner edited it in VS Code), the write is refused with the current version, and the application offers to reload or keep both.
DR-8.7	The service shall accept requests only from the application's own origin and from no origin (local tools), and refuse other origins and hosts (V1's rule).
DR-8.8	The file watcher shall debounce changes by 200 ms and report `{ id, change: "added" | "changed" | "removed" }`; a new item appears within 2 s (FR-6.9.2).
DR-8.9	When the service does not answer, the application shall say so in the status strip and go read-only: every command that writes is greyed with "the Node service is not running (start it with ...)" (FR-6.18.6).

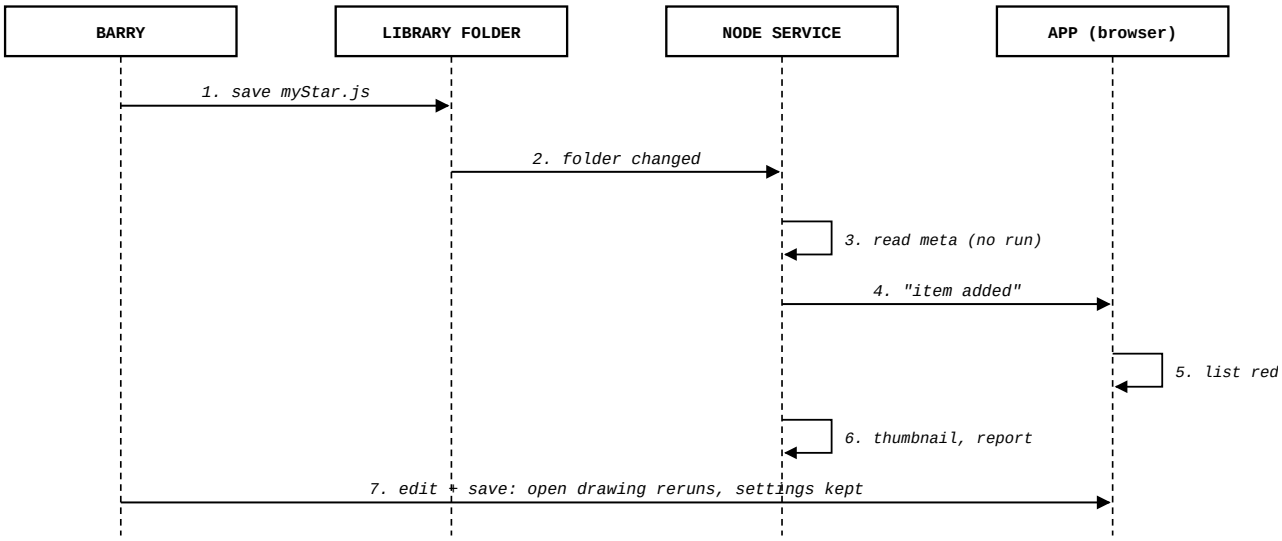


Figure 10. Adding an item by saving a file

8.3 Acceptance tests

AT-8.1	Save a new script in VS Code: it is listed, with a thumbnail, within 2 s; no restart.
AT-8.2	Edit an open figure in VS Code and in the app at once, then save from the app: the conflict is reported, nothing is lost.
AT-8.3	Request a path with `..`, an absolute path, and a foreign origin: all three refused.
AT-8.4	Stop the service: the app says so within 5 s; Save is greyed with the reason; start it: Save returns.

9.0 THE SCRIPT HOST AND THE LEGACY ADAPTER

9.1 Isolation

V1 runs scripts in the application's own page, sharing `ctx`, `drawState` and window globals; two scripts failed after about a hundred earlier runs (P-12; fixed in V1 by a reset, but structurally still shared). V2 runs each script in its own frame.

DR-9.1	Each script shall run in a fresh sandboxed iframe (`sandbox="allow-scripts"`), with its own canvas, given its settings by message and returning either Geometry (a recorded run) or pixels (a painting script). Nothing it does can reach the application.
DR-9.2	A run has a time limit (default 5 s, a setting per script) and a point budget (DR-17.1); past either it is stopped and reported (FR-6.18.2).
DR-9.3	An animated script runs only while its view is on screen and stops when left.
DR-9.4	The host shall record, while a legacy script runs, every line it draws through the adapter's `drawLine` and every canvas stroke, giving a *recorded geometry* that can be placed in a figure as a frozen layer.

9.2 The legacy adapter

The survey of the 185 V1 scripts (R-12) lists what they use: 85 distinct imported names from 29 modules, the `drawState` fields they touch, bare `ctx` in 88 scripts, `scriptInfo` controls with a `redrawHandler` in 80, `armInteractor` in 8, draggable points in 3, animation in 4, blending or transparency in 20, fills or text in 29.

DR-9.5	Inside the frame, the adapter shall provide every one of those 85 names at the module paths the scripts use (both `/draw/x.js` and `../../draw/x.js` forms), implemented over the V2 kernel where a kernel equivalent exists and as V1's own code otherwise, so every V1 script runs unchanged.
DR-9.6	The adapter's `scriptInfo` support shall build the script's controls in the host's settings panel from its V1 declarations (converted by FR-6.4.8) and call its `redrawHandler` on change.
DR-9.7	Every legacy script shall be classified in a migration ledger: runs, adapted, preview only (fills, text, blending: kept as playpen with their own canvas), or blocked, with a reason.

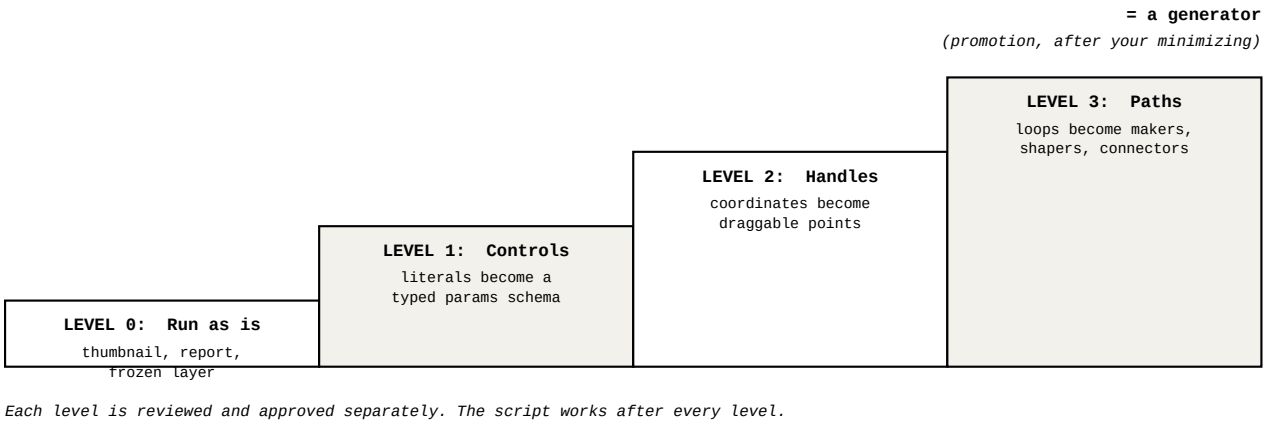


Figure 11. The ingestion ladder: from a foreign script to a generator

9.3 Acceptance tests

AT-9.1	Run all 185 legacy scripts in order and shuffled: every one that ran in V1 runs, and its recorded geometry matches V1's approved copy (FR-6.17.4).
AT-9.2	A script with an endless loop is stopped at its time limit; the app stays responsive.
AT-9.3	A script that writes `window.x = 1` leaves no `x` in the application.

10.0 RENDERING AND OUTPUT

10.1 The canvas renderer

DR-10.1	Paint order: background; layers bottom to top (dimmed when not active, unless Show All or Final); discs; weld marks; magic-point and intersection marks (points mode); then, on a separate canvas, handles, selection, centroid and the drag preview.
DR-10.2	Final is a renderer option: points only, guides and scaffold left out (FR-6.11.2).
DR-10.3	Hit-testing uses the same geometry and transforms the renderer draws, from the evaluation result, never from what was painted.
DR-10.4	Device-pixel ratio is honoured, so lines are sharp on high-density screens; line width is in document units, scaled with zoom.

10.2 Physical output

The output is what the owner hammers from. Its correctness is checked with a ruler.

```
nailsOf(document, evaluation):
  nails = []
  for each disc point (and, if a layer has no discs, each stitch end of a printing layer):
    if the layer's role is guide or scaffold, or includeInOutput is false: skip
    p = its position, in document units, transformed
    add { number: point number, x, y, z, sources: [{ layer, path, i }] }
  merge nails closer than 0.05 mm: one nail, all numbers ("11+12"), all sources   // a weld is
already one point
  return nails

templatePages(nails, scale, paper, margin, overlap):
  convert to mm (x * scale); find the extent; tile it with sheets of (paper - 2*margin),
  each overlapping its neighbour by overlap; on each sheet draw every nail inside it (+ the
overlap),
  a cross at its centre, its number beside it, alignment marks in the overlap;
  page 1: title, scale, nail size, a 100 mm calibration line and a 10 mm scale bar
```

DR-10.5	The printed 100 mm calibration line shall measure 100 ± 0.5 mm when printed at "actual size"; the cover page says to print at actual size and to measure it.
DR-10.6	Nail spacing check: every pair of nails whose centres are closer than diameter + clearance is ringed and listed; it warns, it does not refuse (FR-6.11.6).
DR-10.7	Colour cycle (FR-6.11.9, O-14): when on, a nail's colour is `palette[step mod 5]`, step being its index i along its source path; a nail with several sources is drawn as a circle split into its colours. Off by default.
DR-10.8	Nail list (CSV): number, x, y, z in the chosen units, sources, merged numbers.
DR-10.9	SVG output is the canvas renderer's same paint list written as SVG paths, in mm with a `viewBox`, so it opens at true size in Inkscape and drives a pen plotter or laser cutter.
DR-10.10	Per-layer output (FR-6.11.10): one file per printing layer, the same sheet layout and alignment marks on each, for layers mounted at depth.

10.3 Acceptance tests

AT-10.1	Print the calibration page at actual size on the owner's printer; the line measures 100 mm.

AT-10.2	A figure 600 mm wide on Letter paper with 10 mm margins and overlap tiles to the expected sheet count; every nail appears on exactly the sheets whose area (with overlap) contains it.
AT-10.3	Two welded discs print as one nail with both numbers; two unwelded discs 0.01 mm apart likewise, with a note.
AT-10.4	The SVG of a 100 mm square opens in Inkscape measuring 100 mm.

11.0 THE COMMANDS TABLE AND THE SCREENS

11.1 Three levels, fixed places

FS 6.13. The level of a command is what it acts on. Each level has one place, the same on every tab, so the owner, after a month away, has one rule to remember.

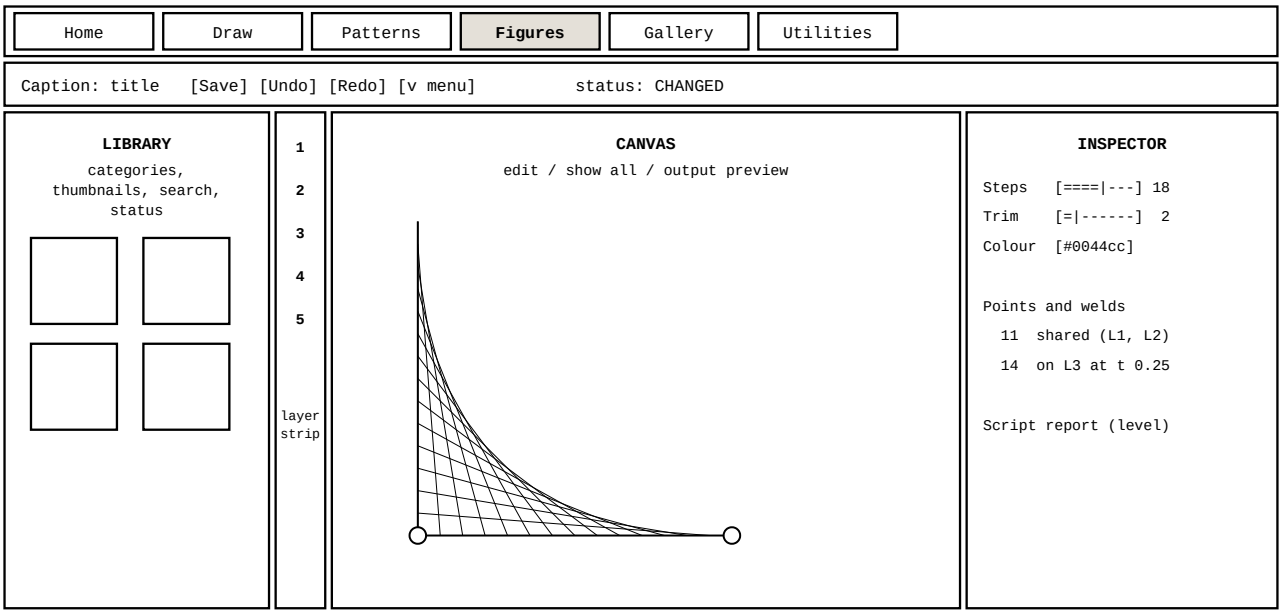


Figure 12. The screen: where each level of command lives

DR-11.1	The menus (global strip, Commands button panel, caption "v" menu, a layer's chip menu) shall be built from the commands table by filtering on `scope` and on the context (active tab, subtab, layer). No screen shall build a menu item by hand.
DR-11.2	Context-aware commands: one `save`, `saveAs`, `export`, `reset`, `help`, `notes`, `history` each, reading the context to decide what they act on. A tab supplies a small adapter (what "the open item" is), not its own command.
DR-11.3	Keyboard: each command may declare a key; keys are shown in the menus; the same key does the same kind of thing on every tab (Ctrl+S save, Ctrl+Z undo, Delete delete the selection).
DR-11.4	Before the table is written, a survey of every V1 command shall be made (FR-6.13.7): id, label, where it is now, which level it belongs to, and duplicates. The survey is reviewed by the owner.

11.2 Acceptance tests

AT-11.1	On every tab, Help is in the same place and opens the page for that tab and subtab.
AT-11.2	Save on Draw, Figures and Patterns is one command id; its greyed tooltip, with nothing open, says "nothing open to save".
AT-11.3	A code search finds no menu item built outside `screens/menus.js`.

12.0 ERROR HANDLING

FS 6.14. The design rule: no checks in the logic; catch at a few choke points; show every error; never stop the whole application (D-12). V1 has this since `claude/v1-errors`; V2 is built with it.

Choke point	On error
A layer's evaluation	That layer draws nothing and shows a red mark; the others draw. The error names the document and layer.
A command's run	The document is unchanged (DR-7.4); the error names the command.
A tab switch	The tab shows a message in its main area; other tabs work.
Reading or writing a file	The item is marked; the error names the file and what was wrong with it.
Registering a generator, template or connector	It is listed greyed with the reason; others load.
A script run (in its frame)	The frame reports it; the script's view shows it; the app is unaffected.
Anything uncaught	The window's error and unhandled-rejection handlers report it as "uncaught".

```
ErrorRecord   = { time, where, message, place, count }     // place: "file:line" of the
owner's code, if any
error log     = journal/logs/errors_YYYY-MM-DD.txt, one line per new error:
  2026-11-02 14:03:11  Figure "star", layer 3 (parabola): steps is not a number
(generators/parabola.js:22)
```

DR-12.1	The error bar is a fixed region of the page, on every tab, not cleared by a tab switch; it lists the latest five different errors with counts, and a Dismiss.
DR-12.2	An error is logged once per page load per (where, message); repeats increment the count shown in the bar.
DR-12.3	An error inside the error handler itself is written only to the console, so a fault in reporting cannot loop.
DR-12.4	At start-up, Home summarises the newest error log (count and latest lines) with its own Dismiss, remembered by file name and count, so new errors in the same file show again.
DR-12.5	Debug checks (FR-6.14.5) are in `documents/checks.js`, run after each command when `session.debug` is on: point numbers unique and below nextPoint; every handle names an existing point; every attached point names an existing layer; order lists every layer once. A failure is reported like any error.

12.1 Acceptance tests

AT-12.1	Make one layer's generator throw: the bar names the layer; the other layers draw; the log has one line; redraw ten times: the count is 11, the log still one line.
AT-12.2	Reload: Home says the newest error log has 1 error.

13.0 EXPLORATION GENERATORS AND GEOMETRY ALGORITHMS

13.1 Lattice with shared points

The `manyRowsColumns` model (FR-6.19.1): M shapes per row, N rows, neighbours sharing points, so one drag reshapes several shapes. Copies are added with a setting.

```
lattice(settings):
  shape: a generator id with k handles; rows N, cols M; spacing dx, dy; shareRule: "edges" |
"corners" | "none"
  lay out an (M+1) x (N+1) grid of point numbers in the document's point table (created once,
kept on resize
    where positions exist, new ones added at the grid position)
  for each cell (r, c): a child layer of the shape whose handles name the grid's point numbers
for that cell,
    chosen by shareRule (corners: the four cell corners; edges: also the edge mid-points)
  the lattice is one layer to the owner (one settings panel), many layers to evaluation
```

DR-13.1	Changing M or N keeps every existing point where it was dragged; new points take grid positions (manyRowsColumns re-seeds and loses drags; V2 shall not).
DR-13.2	A lattice may list several shapes, used in a repeating pattern across the cells, and an overlay layer (one shape resting on top, its own colour) (FR-6.19.2).
DR-13.3	A drag of a shared point is one command and redraws every cell using it in the same frame (DR-6.6).

13.2 Symmetry repeat

Any layer may carry `repeat: { n, center, mirror }`: evaluated once, drawn n times turned 360/n° about the centre (and mirrored if asked). Repeated copies are drawn, not new layers; their nails are real (they print) and numbered after the original's.

13.3 Component groups

A group file lists layers with mostly default settings and their welds; inserting it inserts them all, as one command, with fresh point numbers (FR-6.19.5).

13.4 Roulettes

```
roulette(R, r, d, turns, steps, dChange):
  k = (R - r) / r
  for i in 0..steps: th = turns * 2pi * i / steps;  dd = d + dChange * th / (2pi)
    x = (R - r) cos th + dd cos(k th);  y = (R - r) sin th - dd sin(k th)
  closes after turns = r / gcd(R, r) when R and r are whole numbers; otherwise never: limited
by turns
```

13.5 Templates

FS 6.20. A template is a generator with `meta.kind = "template"`, inserted with role guide: shown while working, left out of Show All, Final and output. The first set: rectangle, circle, ruler (shows its length in the caption, in mm once the scale is set), bounding box of the figure, saw tooth, grid.

13.6 The pursuit engine

One engine for every V1 pursuit script (FR-6.19.6). Prototype: `gallery/Ideabook/scripts/pursuitEngine.js`.

```
pursuitSteps(start: Point[], s):
  defaults: steps 40, rule "next", k 2, move "fraction", f 0.08, speed 4, speeds null,
            bias 0, jitter 0, seed 1, reverse false, center, radius 100, target
  pts = reverse ? start reversed : start;  rnd = seeded(seed);  heading = none for each point
  out = [pts]
  repeat steps times:
    for each point p_i:
      t = target by rule:  next p_(i+1) | kth p_(i+k) | opposite p_(i+n/2) | centroid of all
                           | random (a target fixed per point from rnd) | circle (a point
moving on a circle)
                           | fixed (s.target) | a function (i, pts, step)
      d = t - p_i
      rate = speeds ? speeds[i] : (move = "speed" ? speed : f)
      d = move = "speed" ? unit(d) * min(rate, |d|) : d * rate
      d = d * (1 - bias) + heading_i * bias;   heading_i = d
      p_i' = p_i + d + jitter * (rnd() - 0.5) * 2 (each axis)
    pts = all p'; out.push(pts)
  return out                                      // drawn as "edges" (nested polygons) or
"trails" (each point's path)
```

V1 script (gallery/Scripts/Pursuit/)	Engine settings that reproduce it
pursuit.js	rule next, move fraction, cw or ccw by reverse; as edges
pursuitMice.js	the same, start a regular n-gon
pursuit2.js	rule next | kth | centroid; speeds from a seed for its "speeds" rule
pursuit3.js	rule next, as trails
pursuitStatic5 / Dynamic5.js	several runs, some reversed
copilot1 / copilotAnimation.js	move speed, bias, jitter, rule random | opposite | circle | fixed; animation = drawing steps over time
pusuitSurface.js	not covered: it fills polygons (playpen)
pursuit4.js	not a pursuit: a stitch rule (pairwise with offset 2 sides)

13.7 Intersections

Intersections are nail candidates, shown with their own mark, distinct from magic points (FR-6.2.7a, 6.21).

```
intersections(pathA, pathB, s):
  segments of A and B (a closed path includes its closing segment)
  candidate pairs by a uniform grid of cell size = 2 x the mean segment length (avoids n*m)
  for each pair, solve the two-segment intersection; keep 0 <= u, v <= 1
  merge points closer than 0.5 px
  name each "cross A.B #k", k numbered by angle about the pair's joint centroid from straight
up, clockwise (DR-4.9)
tracking: a disc attached to "cross A.B #k" is re-found after a change by name; if #k no
longer exists,
  the nearest crossing to its last position within 5 px is taken and the change noted;
otherwise it is marked lost (DR-6.3)
```

DR-13.4	Self-intersections only on request (DR-4.11). At most 400 intersections per pair are offered (V1's cap), the rest counted.
DR-13.5	Later (FR-6.21.2), an intersection may be defined by rule: "the k-th crossing of A and B counted from A's start", which is stable where angle numbering is not.
DR-13.6	The owner's own intersection routine shall be studied first: for which curves it works and why it fails on others, and kept if it is better on its curves.

13.8 Image to thread (later)

FR-6.19.7, for analysis now, building later: approximate a photograph with one continuous thread across N nails on a circle. The known greedy method: start at a nail; repeatedly choose the next nail whose straight line most darkens the remaining darkness of the image along it; subtract the line; stop after L lines or when no line helps. Output is a nail sequence, the build instructions in this one case.

13.9 Acceptance tests

AT-13.1	A 3 × 2 lattice of Parabolas sharing corners: drag a shared corner, all cells using it follow; change to 4 × 2: the dragged corner stays.
AT-13.2	Each V1 pursuit script listed as covered is reproduced by the engine within 1 px per point for its default settings.
AT-13.3	Two crossing lines give one intersection named "cross L1.line.L2.line #1"; a disc on it follows as one line turns 10°.

14.0 JOURNALS, RUN HISTORY, NOTES AND LEARN

14.1 The journal file

One journal per item, beside documents and never inside them (D-10). One JSON object per line (JSON Lines), added and never rewritten, readable in VS Code.

```
journal/scripts/curve-stitch/manyRowsColumns.jsonl
{"time":"2026-11-
02T14:03:11Z","kind":"run","run":"r00041","source":"runs/r00041.js","picture":"runs/r00041.png
","changed":"+3 -1 lines"}
{"time":"2026-11-02T14:05:40Z","kind":"note","text":"Offset 3 makes the middle rows twist.
Keep.","about":"r00041"}
{"time":"2026-11-02T14:06:02Z","kind":"mark","run":"r00041"}
{"time":"2026-11-02T15:20:00Z","kind":"study","study":"s0007","file":"studies/s0007.json"}
{"time":"2026-11-02T15:31:12Z","kind":"explanation","about":
["r00041","r00042"],"text":"...","approved":true}
{"time":"2026-11-03T09:00:00Z","kind":"commit","hash":"9f6040d","runs":["r00041","r00043"]}
```

DR-14.1	A journal's folder holds its runs (`runs/rNNNNN.js` and `.png`) and studies (`studies/sNNNN.json`). Run and study numbers are per item, never reused.
DR-14.2	Appends go through the service (`journal.append`), which writes the line with a single append call, so a crash cannot damage earlier lines; a torn last line is ignored on read.
DR-14.3	Journals are in git like everything else, except `runs/` pictures older than the retention period, which may be thinned (DR-14.6).

14.2 Run history

```
on each run of a script (or generator, from Draw):
  text = the source as loaded;  h = hash(text)
  if h == the hash of this item's last run: nothing new; done
  number = next run number
  write runs/rN.js = text
  after the first frame: write runs/rN.png = a 240 px picture of the drawing
  append { kind: "run", run: rN, changed: line diff counts against the last run, error?: the
run's error }
```

DR-14.4	Capture is at run time (P-23): every run whose source changed, whether or not the owner saved in VS Code, committed, or marked anything.
DR-14.5	The history panel lists runs newest first with their pictures; for any run: Run (draw that version, file untouched), Diff (with the run before), Against now, Restore (replace the file's text, itself a recorded change), Mark, Note.
DR-14.6	Retention (O-15): unmarked runs older than the period (default 30 days) are thinned to the first and last of each working session (runs less than 30 minutes apart are one session). Marked runs, and runs a commit entry names, are kept for good.
DR-14.7	An old run is executed against today's kernel and draw code; the panel says so, and shows the stored picture beside the new one, so a difference is visible (V1's Script history cannot show the stored picture).

14.3 Notes

One global Notes command opens the journal of whatever is open (script, generator, figure, or the active layer by `lid`) with a box to add a note; notes show in the item's history and in the Notes popup beside the header comment.

14.4 Learn

```
Learn request (only when the owner presses for it):
  what:  { kind: "runs", a: rN, b: rM }         the code diff and both pictures
       | { kind: "regression", item, change }    the approved and new pictures and geometry
summary, and the diff of the code since approval
       | { kind: "ideabook", image }             the image, and the routines catalogue: "draft
a hint"
  shown first: exactly what will be sent, and to which service (O-13); the owner presses Send
or Cancel
  reply: a draft explanation or draft script, shown in an editor; the owner edits, then
Approve or Discard
  Approve: appends { kind: "explanation", approved: true } (or saves the draft script as a new
item, never over an existing one)
```

DR-14.8	Nothing leaves the machine without the owner's press, every time; there is no automatic or background Learn (D-13).
DR-14.9	Nothing Learn returns is written to a journal or the library until approved. Discarded drafts are not kept.
DR-14.10	A Learn draft script is a hint (section 16.4) or a new playpen script; it never replaces a file the owner wrote.

14.5 Acceptance tests

AT-14.1	Run a script, change one line in VS Code, save, run: the history has two runs, the second's Diff shows the one line, both pictures shown.
AT-14.2	Kill the service mid-append: the journal reads back with every complete line.
AT-14.3	Learn with no network: the request is refused with a clear message; nothing is written.

15.0 SWEEP

FS 6.15. A parameter study in two forms, both acting on the open object (Draw) or the active layer (Figures).

```
values(setting, from, to, count):
  base = setting.min if defined else 0;  step = setting.step (int: 1)
  for i in 0..count-1: raw = from + (to - from) * i / (count - 1)
    v = base + round((raw - base) / step) * step;  clamp to [min, max];  drop repeats
grid (form 1):     one setting: up to 16 values, 4 across; two settings: values1 across,
values2 down, at most 16 cells
slideshow (form 2): frames = one-setting run, or the two-setting grid in an order (O-12):
                      "rows" (setting 1 changes fastest) or "snake" (alternate rows reversed,
so neighbours differ by one step)
state machine:     Playing --Pause--> Paused --Play--> Playing;  Paused --Back--> previous
frame (stays Paused)
                   Playing --(frame time 5..10 s)--> next frame;  any --Stop--> Stopped
(applies the frame shown)
                   clicking a grid cell applies that cell
```

DR-15.1	Pictures are made off screen by the renderer (FR-6.11.8), from private evaluations; the open document is not changed until a choice is applied.
DR-15.2	Applying is one command ("Sweep: steps 24, offset 3"), undoable.
DR-15.3	The seed is held fixed across a sweep of a random generator (FR-6.15.4).
DR-15.4	Frame time is a setting, 5 to 10 s, default 7 s; Back during Playing pauses and goes back one frame.
DR-15.5	Save as study (FR-6.16.7): the study file holds target, starting settings (a copy), the swept settings and their values, form, order, frame time, seed and the chosen cell; replay offers "as it was" and "as it would look now".

15.1 Acceptance tests

AT-15.1	Sweep Cycloid's factor 2 to 5 in 4 pictures and nodes 60 to 120 in 4: a 4 × 4 grid labelled with both values; click a cell: the object takes both values; Undo restores them.
AT-15.2	Slideshow at 5 s: Pause holds; Back shows the previous frame; Stop applies the frame shown.
AT-15.3	A saved study replayed "as it was" after the figure has changed shows the original pictures.

16.0 REGRESSION, CHECKS, ROBUSTNESS AND IDEABOOK HINTS

16.1 The regression harness

V1 has one (`npm run test:regression`, 199 items); V2 is built with it from the first checkpoint (D-9).

```
for each item (generator at defaults, saved figure, legacy script, Ideabook hint):
  evaluate (or run in the host) twice
  geometry = every stitch's two ends, rounded to 0.01 document units, per layer, in order
  fingerprint = sha1(geometry text);  picture = 240 px PNG
  stable = both runs give the same fingerprint (and, with no geometry, the same picture)
compare with tests/regression/approved.json + approved/<item>.png:
  no approved copy: listed as "not yet approved", does not fail
  unstable: listed, not compared
  geometry present: changed if the stitch count or fingerprint differs
  geometry absent (a painting script): changed if more than 0.1 % of pixels differ by more
than 8 levels
  error status changed: changed
report: tests/regression/report.html, approved and new pictures side by side, changes only
accept [name]: replace the approved copy of matching items (all if no name); commit
tests/regression/
```

DR-16.1	The harness is one command and runs before every review and at every checkpoint; nothing merges with an unexplained change (FR-6.17.2).
DR-16.2	For the V1-to-V2 comparison (FR-6.17.4), geometry is compared within a tolerance of 0.5 px per point after aligning bounds, since exact reproduction is not required (D-4); the report lists items outside it for the owner's eye.
DR-16.3	Approved copies made on one platform may differ in pictures on another (anti-aliasing, fonts); geometry is the deciding comparison; pictures only for items with no geometry, and a platform's pictures can be re-accepted wholesale.

16.2 Other checks

Check	What it does	When
lint	no implicit globals (V1's lint:globals)	every commit (hook)
imports	every import resolves; no layer imports from above (DR-3.1)	every commit
removed code	exported names before and after; prints what disappeared (I-8)	every commit, every review
ids	every element id looked up exists in the page or is created by code	every review
schemas	every generator, template, connector registers cleanly	start-up and every review
documents	every saved figure passes the integrity check	every review
kernel	pure-function tests (section 4.8), no browser	every commit
service	every request, including refusals (V1's test:node)	every review
screens	tab switches, the commands of each level (V1's test:tabs)	every review

16.3 Robustness

DR-16.4	Budget (FR-6.18.2): an evaluation may produce at most 500,000 stitches and take at most 2 s of compute per layer (settings); past either it stops, the layer shows "stopped: 1,240,000 stitches asked for; the setting *nodes* (2,000) is the likely cause", and the rest of the figure draws.
DR-16.5	Evaluation of a layer may run in a Web Worker when it is expected to exceed 50 ms (by its last time), so the page stays responsive; results are the same either way.
DR-16.6	Kept versions (FR-6.18.4): on each save the previous file is kept as `<name>.v1.json` … `.v5.json`, the oldest dropped.
DR-16.7	Integrity check with repair (FR-6.18.5): listed problems each offer a repair (drop the dangling reference, re-attach to the nearest feature, renumber), applied as one undoable command.

16.4 Ideabook hints

DR-16.8	A hint is a document (FR-6.22.2) stored as `library/ideabook/<category>/hints/<image name>.json`, with a header note: what the image seems to be, what it is built from, what was guessed, and, for a numbered Figure, the StringThing function that drew it.
DR-16.9	The link is by name; characters the server will not serve in a file name (`& , # ? % + ;`) are written as _ (V1's rule).
DR-16.10	The Ideabook view shows the image and the hint side by side when the window is wide enough, toggled otherwise; Open as figure copies the hint into a new figure.
DR-16.11	V1's 282 hint scripts are converted by the importer (FS 6.10) as its first large test; each converts or is listed with why not.

17.0 NON-FUNCTIONAL REQUIREMENTS

DR-17.1	Performance, on the owner's ordinary Windows PC: dragging a handle in a 20-layer, 20,000-stitch figure redraws at 60 frames a second; opening a 100-layer figure takes under 2 s; a full regression run of 1,000 items takes under 10 minutes.
DR-17.2	Start-up to a usable Home under 3 s with 2,000 catalogue items (no start-up rebuild: P-10).
DR-17.3	Readability (DR-2.5) is a requirement, not a preference: every file has its header comment; a reviewer unfamiliar with the code can say what a file does from its header and its first screen.
DR-17.4	Dependencies: Vite (development only), Playwright (tests only), and in `vendor/` only what a feature cannot do without (d3-delaunay for Voronoi; Acorn for the importer, O-4). Each is pinned and listed with why.
DR-17.5	Browsers: current Chrome and Edge on Windows; Safari on iPad for review (read-only).
DR-17.6	Accessibility at the owner's level: every control has a visible label; keyboard use for the common commands (DR-11.3); colour is never the only signal (greyed items also say why).
DR-17.7	Security is local: the service listens on localhost only, checks origins (DR-8.7), refuses paths outside the library (DR-8.4); Learn is the only outbound connection and is the owner's press each time (DR-14.8).

18.0 MIGRATION FROM VERSION 1

18.1 Figure files (format 1)

V1's figure format is fixed and documented (R-11) and may only gain fields, so every V1 figure can be read.

V1 (format 1)	V2 (format 2)
`layers[i]` with `id`, `lid` "L"+(i+1), `objectVersion`, `role`, `includeInOutput`, `visible`, `deleted`, `color`, `params`, `transform`	layer with the same `lid`, role, transform, output and visibility; `generator` = V1 `id`; settings = the generator's "from V1" mapping of `params` keyed by `objectVersion`; deleted layers dropped (their numbers stay used)
`params.points` and named fields (`pt1` ...)	handle points: each a new point number, from the named fields (preferred over the `points` mirror)
points: `anchored` {layer, arm, t, feature?}	attached point {on: lid, path: arm, t} or {on, feature}
points: `free` {ox, oy}	free point at (ox, oy) transformed by the layer
points: `handle` {h, hname}	the handle's point (by name)
`welds` [{members}]	members that are free or handle discs become one shared point; a group with an anchored disc becomes an attachment to that disc
`units` "px", `scale` mm/px, `nail`	`units` "legacy", `scale` carried, `nail` carried
`drawOrder` (indices, top first)	`order` (lids, top first)

18.2 Draw objects

Each of the 14 V1 registry objects is rewritten as a generator (D-5) after a one-page feature list agreed with the owner (FR-6.3.9), with a "from V1" settings mapping. Known particulars: Parametrics gained handles (centre, size) on `v1-finish`; Linked Circles strokes its circles itself (O-16); Circular Parabola (Multi) and Linked Circles are kept for the sketching purpose, not because a template needs them.

18.3 Scripts, hints and approved copies

DR-18.1	Every V1 pattern and gallery script runs in the host unchanged (DR-9.5); its recorded geometry is compared with V1's approved copy (FR-6.17.4).
DR-18.2	V1's approved regression copies (`tests/regression/`) are the baseline for that comparison; they are made on the owner's PC before cut-over.
DR-18.3	The 282 Ideabook hint scripts convert to hint documents (DR-16.11); their shared routines map to kernel connectors and composites (FR-6.2.5a, b).
DR-18.4	Saved Draw variations ("secondary objects") import through the same mapping as layers.
DR-18.5	V1 remains usable throughout (D-1); cut-over is the owner's decision at CP-8.

19.0 CONSTRUCTION AND TEST PLAN

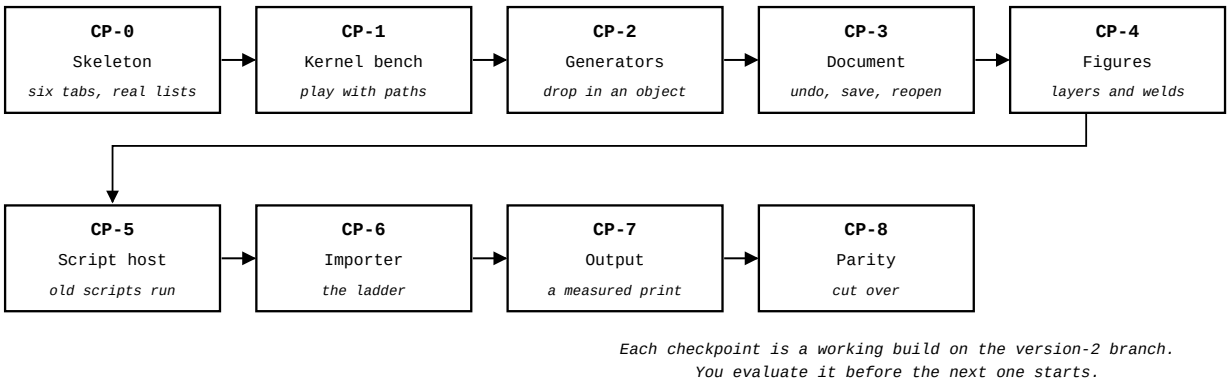


Figure 13. Checkpoints and their order

Each checkpoint is a branch the owner reviews, uses and accepts before the next begins (FS 10). The table lists, for each, what is built, the acceptance tests of this document that must pass, and the regression state expected.

CP	Built	Must pass	Regression
CP-0 Skeleton	folders and layers (3.2), the service with its core requests, catalog from folders, error bar and log, the regression harness, the commit hook and checks, plugin finding, seeds, z in points	AT-8.1, 8.3, 8.4, 12.1, 12.2	harness runs on V1's approved copies (nothing V2 yet)
CP-1 Kernel bench	kernel types, makers, shapers, connectors, composites, finders, pursuit engine, roulettes, adaptive sampling; the kernel bench page	AT-4.1 to 4.7, 13.2	kernel items approved
CP-2 Generators	generator files and schema, the common settings panel, controls, the 14 rewritten objects, commands table with scopes, journals and Notes, the budget	AT-5.1 to 5.3, 11.1 to 11.3, 16.x schema check	generators approved; V1 Draw comparison reviewed
CP-3 Documents	document model, point table, welds, attachments, evaluation, commands and undo, session, crash recovery, kept versions, integrity check; Sweep	AT-6.1 to 6.4, 7.1 to 7.3, 15.1 to 15.3	saved test figures approved
CP-4 Figures	the Figures tab, discs, points mode, magic points and intersections, transforms, lattice, symmetry, templates	AT-13.1, 13.3	V1 figures imported and compared
CP-5 Script host	the frame, adapter, recorder, limits, run history with pictures, Learn	AT-9.1 to 9.3, 14.1 to 14.3	all 185 legacy scripts compared
CP-6 Importer	ingestion levels 1 to 3, review panel; Ideabook hints as documents	DR-16.11 ledger complete	hints approved
CP-7 Output	SVG, PDF with tiling and calibration, nail list, spacing check, colour cycle, per-layer files	AT-10.1 to 10.4	output files compared
CP-8 Parity	everything V1 did that the owner wants; help pages; cut-over	every AT; the owner's own walk through every tab	no unexplained change anywhere

DR-19.1	Each checkpoint ends with: every check of 16.2 green; the regression report reviewed with the owner; a checklist for the owner of what to try by hand (V1's CHECKLIST files are the model); and the decisions log updated.

DR-19.2	Guesses made while building go in a decisions log with the guess and how to reverse it (V1's `V1_FINISH_PLAN.md` G-1 to G-18 is the model).

20.0 RISKS AND QUESTIONS FOR ANALYSIS

20.1 Risks

No.	Risk	Likelihood / effect	Mitigation in this design
K-1	The rebuild takes long enough that V1 and V2 drift, and the owner maintains two programs.	High / high	Checkpoints each usable on their own; V1 frozen except fixes; regression copies tie the two; cut-over only at CP-8.
K-2	The kernel's abstraction (paths, connectors) does not fit some of the owner's playful objects, and they lose what made them interesting.	Medium / high	Feature lists agreed per object before rewriting (FR-6.3.9); playpen scripts stay as scripts; Circular Parabola (Multi) and Linked Circles named as tests of the kernel.
K-3	Readability is lost to generality: a kernel of many small parts is harder to follow than V1's one-file-per-object.	Medium / high	DR-2.5, DR-5.5, DR-17.3; a generator fits on a screen; kernel parts are named for what they draw.
K-4	Evaluation is too slow for dragging large figures without a framework's tricks.	Medium / medium	Per-layer memoisation (DR-6.6), workers (DR-16.5), budgets (DR-16.4); measured from CP-1.
K-5	Snapshot undo uses too much memory on large figures.	Low / medium	Structural sharing; limits in DR-7.2.
K-6	Feature names (tips, intersections) renumber unexpectedly, and discs jump.	Medium / high for templates	Angle numbering (DR-4.9); tracking with a 5 px rule and a "lost" state (13.7, DR-6.3); rule-based definitions later (DR-13.5).
K-7	The iframe host breaks scripts that reach into the page in ways the survey missed.	Medium / low	The ledger (DR-9.7); preview-only and blocked are legitimate outcomes.
K-8	An AI assistant building V2 repeats past failures: deleting unrelated code, shortcutting existing patterns.	High / high	The removed-code check on every commit; layer import check; the owner's review per branch; this document as the pattern to follow.
K-9	Learn leaks work or invites unreviewed changes.	Low / medium	DR-14.8 to 14.10.
K-10	True-scale output is wrong on a particular printer.	Medium / high	Calibration line on every print (DR-10.5); SVG for a known-good path.

20.2 Questions for analysts

The owner would most value an outside view on these. Each names the sections it concerns.

No.	Question	Sections
Q-1	Is "paths, shapers, connectors, finders" the right kernel? What common string-art constructions would it express badly?	4, 13
Q-2	Is a weld as a shared point number, with attachments as one-way references, sufficient, or will the owner need two-way constraints (a point on two curves at once)?	6
Q-3	Is whole-document snapshot undo adequate at the stated scale, or should commands record inverse operations?	7
Q-4	Can per-layer memoisation and workers meet DR-17.1 in plain JavaScript, or is a different drawing approach (WebGL, OffscreenCanvas) needed for large figures?	6.2, 16.3, 17
Q-5	Is the iframe host the right isolation for legacy scripts, versus a Worker with a canvas proxy, given that 88 scripts paint with a bare context?	9
Q-6	Is "folders are the truth" with small metadata files robust on Windows with VS Code editing the same files (watcher reliability, partial writes, case-insensitive names)?	8

Q-7	How should feature names stay stable as shapes change? Is angle numbering plus tracking enough, or is a different identity (topological, by construction) needed?	4.7, 13.7
Q-8	Is the regression design (geometry fingerprints, picture fallback, accept per item) the right balance of strictness and noise?	16.1
Q-9	What does the design make hard that the owner will probably want next (3D boards, plotters, image to thread, layered mounting)?	10, 13.8
Q-10	Where is the design over-built for one person's hobby tool, and what could be dropped without loss?	all
Q-11	Is the checkpoint order right? Would a different order give the owner something usable sooner?	19
Q-12	Readability: does any part of this design force code the owner will find hard to read, and is there a plainer way?	2.3, 5, 17

20.3 What an analyst should not need to settle

The open items of FS 12 (O-1 to O-16) are the owner's decisions: names, sizes, colours, retention, which AI service. An analyst may comment, but the design is written to work with any answer.

21.0 REQUIREMENT INDEX

Each FS requirement area, and where this document details it.

FS	Subject	This document
6.1	General	2.3, 3, 17
6.2	Kernel	4
6.3, 6.4	Generators, settings	5
6.5	Documents, point table, welds	6
6.6, 6.7	Commands, undo, session	7
6.8, 6.9	Catalog, storage, adding by saving	8
6.10	Script host, ingestion	9
6.11	Rendering, output	10
6.12, 6.13	Menus, commands at three levels	11
6.14	Errors	12
6.15	Sweep	15
6.16	Journals, Learn	14
6.17, 6.18	Regression, robustness	16.1 to 16.3
6.19 to 6.21	Exploration, templates, intersections	13
6.22	Ideabook hints	16.4
10, 11	Construction, verification	19, and the AT- tables throughout

END OF DOCUMENT